

Objective: Find the shortest road-distance route between two Indian cities using Dijkstra's algorithm.

The graph represents cities as nodes and road distances as weighted edges. The following is a sample network of major Indian cities, not a complete nationwide road database.

```
import heapq

graph = {
    "Hyderabad": {
        "Bengaluru": 570,
        "Chennai": 626,
        "Mumbai": 710,
        "Nagpur": 500,
        "Vijayawada": 275
    },
    "Bengaluru": {
        "Hyderabad": 570,
        "Chennai": 346,
        "Pune": 840,
        "Mysuru": 145
    },
    "Chennai": {
        "Hyderabad": 626,
        "Bengaluru": 346,
        "Vijayawada": 450,
        "Kochi": 690
    },
    "Mumbai": {
        "Hyderabad": 710,
        "Pune": 150,
        "Ahmedabad": 530,
        "Goa": 590
    },
    "Pune": {
        "Mumbai": 150,
        "Bengaluru": 840,
        "Goa": 450
    },
    "Vijayawada": {
        "Hyderabad": 275,
        "Chennai": 450,
        "Visakhapatnam": 350
    },
    "Nagpur": {
        "Hyderabad": 500,
        "Bhopal": 350,
        "Raipur": 300
    },
    "Ahmedabad": {
        "Mumbai": 530,
        "Jaipur": 660
    },
    "Goa": {
        "Mumbai": 590,
        "Pune": 450,
        "Bengaluru": 560
    },
    "Mysuru": {
        "Bengaluru": 145
    },
    "Kochi": {
        "Chennai": 690
    },
    "Visakhapatnam": {
        "Vijayawada": 350
    },
    "Bhopal": {
        "Nagpur": 350
    },
    "Raipur": {
        "Nagpur": 300
    },
    "Jaipur": {
        "Ahmedabad": 660
    }
}
```

```

        "Jaipur": 600
    },

    "Raipur": {
        "Nagpur": 300
    },

    "Jaipur": {
        "Ahmedabad": 660,
        "Bhopal": 600
    }
}

def dijkstra(graph, start, goal):

    distances = {city: float('inf') for city in graph}
    distances[start] = 0

    parent = {city: None for city in graph}

    priority_queue = [(0, start)]

    visited = set()

    while priority_queue:

        current_distance, current_city = heapq.heappop(priority_queue)

        if current_city in visited:
            continue

        visited.add(current_city)

        if current_city == goal:
            break

        for neighbor, weight in graph[current_city].items():

            new_distance = current_distance + weight

            if new_distance < distances[neighbor]:

                distances[neighbor] = new_distance
                parent[neighbor] = current_city

                heapq.heappush(
                    priority_queue,
                    (new_distance, neighbor)
                )

    if distances[goal] == float('inf'):
        print("No route found")
        return

    path = []
    city = goal

    while city is not None:
        path.append(city)
        city = parent[city]

    path.reverse()

    print("\nShortest Route:")
    print(" -> ".join(path))

    print("Total Distance:", distances[goal], "km")
    print("Visited Cities:", len(visited))

start = input("Enter starting city: ")
goal = input("Enter destination city: ")

if start in graph and goal in graph:
    dijkstra(graph, start, goal)
else:
    print("City not found in the graph")

```

Enter starting city: Hyderabad
Enter destination city: Begaluru
City not found in the graph

